## 1. Transistors & PN Junctions (The Building Blocks)

At the absolute lowest level, a computer only understands two things: **Electricity is ON (1)** or **Electricity is OFF (0)**. The device that controls this is the **transistor**.

- **What is it?** A microscopic electronic switch that can turn current on or off without any moving parts. Modern CPUs have billions of them.
- **The PN Junction (How it works):** Transistors are made of semiconductors (like Silicon).
    - **P-type material:** Has a positive charge (has "holes" where electrons should be).
    - **N-type material:** Has a negative charge (has extra electrons).
    - When you stick them together, it creates a **PN Junction**. By applying a small electrical voltage to a control pin (the "Gate"), you can force the electrons to bridge the gap and flow across the junction (Switch is ON = 1). Remove the voltage, the flow stops (Switch is OFF = 0).

## 2. How Logic Gates are Formed

A single switch (transistor) isn't very smart. But when you wire them together, they can make logical decisions. These are called **Logic Gates**.

- **AND Gate:** Wire two transistors in **series** (one after the other). Electricity only flows through to the end if Transistor A **AND** Transistor B are both turned ON.
- **OR Gate:** Wire two transistors in **parallel** (side by side). Electricity flows through if Transistor A **OR** Transistor B is turned ON.
- **NOT Gate (Inverter):** Wires a transistor so that applying current actually breaks the circuit, turning a 1 into a 0.

By chaining thousands of these AND, OR, and NOT gates together, you can create circuits that add numbers, compare values, and store memory.

## 3. How a Computer Works Internally

Almost all modern computers use the **Von Neumann Architecture**. It consists of three main systems interacting in a continuous loop:

1. **CPU (Central Processing Unit):** The "brain" that executes instructions.
2. **Memory (RAM):** The "fridge" holding the data and code the CPU is currently working on. It loses data when power goes out (volatile).
3. **I/O (Input/Output):** How the computer talks to the outside world (Keyboard, Screen, Hard Drive/SSD).

**The CPU Cycle (Fetch-Decode-Execute):**

To run your code, the CPU performs this loop billions of times a second:

- **Fetch:** Grabs the next instruction from Memory.
- **Decode:** Figures out what the instruction means (Add? Move? Jump?).
- **Execute:** Performs the action and writes the result back.

## 4. CPU Architecture (ALU, Registers, Cache)

If the computer is a commercial kitchen, the CPU is the Head Chef. Here is how the chef's tools break down:

- **ALU (Arithmetic Logic Unit):** This is the calculator inside the CPU. It handles all the math (addition, subtraction) and logical comparisons (is $x > y$?).
- **Registers:** These are the chef's hands. They are the smallest, fastest storage locations located *directly* inside the CPU core. The CPU can only perform math on data that is currently sitting in its registers.
- **Cache (L1, L2, L3):** This is the chef's countertop. RAM is massive but relatively slow to access. To avoid waiting for RAM, the CPU keeps copies of the most frequently used instructions and data in the Cache.
    - **L1 Cache:** Extremely small, extremely fast, inside the core.
    - **L2/L3 Cache:** Slightly larger, slightly slower, shared among CPU cores.

# SDE Interview Q&A

**Q: Explain how a high-level if statement in code relates to physical hardware.**
**Answer:** "When I write an if statement in C++, the compiler translates it into machine code (binary). That binary represents electrical signals. Inside the CPU, those signals control transistors, which form logic gates. The ALU uses these logic gates to physically compare two values, and based on whether the output is a 1 or a 0, the CPU's program counter updates to fetch the next appropriate instruction."
**Q: What is the difference between a Register, Cache, and RAM?**
**Answer:** "It comes down to speed, size, and proximity to the CPU. Registers are the fastest and hold the exact data the CPU is calculating *right now*. Cache is slightly slower but larger, acting as a staging area for frequently used data to prevent bottlenecks. RAM is much larger and holds all currently running applications, but fetching from RAM is significantly slower than fetching from Cache."
**Q: Why do we use Cache? Why not just make RAM faster or Registers bigger?**
**Answer:** "It is a physical and economic limitation. The memory that makes up registers and L1 cache (SRAM) is incredibly fast but physically large and very expensive to manufacture, so we can only fit a tiny amount on the CPU chip. RAM uses a different, cheaper technology (DRAM) that allows for massive capacity but is slower to access. Cache bridges that speed gap."
**Q: What exactly does the ALU do?**
**Answer:** "The Arithmetic Logic Unit is the computational heart of the CPU. While the control unit handles fetching and routing data, the ALU is responsible for actually executing mathematical operations like addition, and bitwise logical operations like AND, OR, and XOR."

---

## 1. Endianness (Big-Endian vs. Little-Endian)

Endianness simply refers to the **order in which bytes are stored in computer memory**.

Imagine you have a 32-bit integer (which takes up 4 bytes of memory). Let's use the hexadecimal number 0x12345678.

- 12 is the Most Significant Byte (the "biggest" part of the number).
- 78 is the Least Significant Byte (the "smallest" part of the number).

How does the computer place these 4 bytes into sequential memory slots?

- **Big-Endian:** Stores the "Big" end first.
    - Memory Order: 12 -> 34 -> 56 -> 78
    - *Analogy:* Just like reading English, left to right.
    - *Where it's used:* This is the standard for **Network Protocols** (TCP/IP). When sending data over the internet, it is almost always sent in Big-Endian format.
- **Little-Endian:** Stores the "Little" end first.
    - Memory Order: 78 -> 56 -> 34 -> 12
    - *Analogy:* Reading backwards.
    - *Where it's used:* Almost all modern computer processors (Intel, AMD x86 architecture) store data this way in RAM.

**Interview Cheat Code:** If they ask why this matters, mention that when a Little-Endian computer sends data over a network, the software has to convert it to Big-Endian (Network Byte Order) before transmitting it, otherwise the receiving computer will read the data backward.

## 2. Logic Gates & Bitwise Operations

You don't need to know how to physically build these, but you do need to know their software equivalents (Bitwise operators in C++/Python).

- **AND (&):** Output is 1 *only* if both inputs are 1. (Used for "masking" bits to zero).
- **OR (|):** Output is 1 if *at least one* input is 1. (Used for setting specific bits to 1).

- **XOR (^):** Output is 1 if the inputs are *different* (one is 1, the other is 0). (Used heavily in cryptography and finding missing numbers in arrays).

- **NOT (~):** Flips the bits (1 becomes 0, 0 becomes 1).

---

This entire concept relies heavily on **De Morgan's Laws**, which are the mathematical backbone of digital logic:

- NOT (A AND B) = (NOT A) OR (NOT B)

- NOT (A OR B) = (NOT A) AND (NOT B)

## 1. Building Gates Using Only NAND

A NAND gate outputs 0 only when both inputs are 1. Its basic logic is NOT (A AND B).

- **NOT Gate using NAND**
    - **How it works:** Tie both inputs of the NAND gate together so they receive the exact same signal.
    - **Logic:** A NAND A
    - **Result:** NOT A
- **AND Gate using NAND**
    - **How it works:** An AND gate is just a NAND gate with its output inverted. You use one NAND gate to do the AND/NOT operation, and a second NAND gate (configured as a NOT gate) to flip it back.
    - **Logic:** (A NAND B) NAND (A NAND B)
    - **Result:** A AND B
- **OR Gate using NAND**
    - **How it works:** You must first invert both inputs individually, then pass those inverted signals into a third NAND gate.
    - **Logic:**
        1. Create NOT A (using A NAND A).

2. Create NOT B (using B NAND B).
3. Pass those into a third gate: (NOT A) NAND (NOT B)

- **Result:** A OR B

## 2. Building Gates Using Only NOR

A NOR gate outputs 1 only when both inputs are 0. Its basic logic is NOT (A OR B).

- **NOT Gate using NOR**
  - **How it works:** Exactly like the NAND configuration, tie both inputs of the NOR gate together.
  - **Logic:** A NOR A
  - **Result:** NOT A
- **OR Gate using NOR**
  - **How it works:** An OR gate is simply a NOR gate with its output inverted.
  - **Logic:** (A NOR B) NOR (A NOR B)
  - **Result:** A OR B
- **AND Gate using NOR**
  - **How it works:** You must first invert both inputs individually, then pass those inverted signals into a third NOR gate.
  - **Logic:**
    1. Create NOT A (using A NOR A).
    2. Create NOT B (using B NOR B).
    3. Pass those into a third gate: (NOT A) NOR (NOT B)
  - **Result:** A AND B

---